PathFinder — Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification document provides a complete description of all functional and non-functional requirements for the PathFinder platform. It serves as a contract between stakeholders and the development team, defining exactly what the system must do.

1.2 Document Conventions

- **SHALL**: Mandatory requirement.
- **SHOULD**: Recommended but not mandatory.
- **MAY**: Optional feature.
- **P0**: Critical (must have for MVP)
- **P1**: Important (needed for complete experience)
- **P2**: Nice to have (future enhancement)

1.3 Intended Audience

- Software Developers
- Quality Assurance Engineers
- Project Managers
- Academic Evaluators

1.4 System Overview

PathFinder is a client-server web application with a React.js frontend communicating with a Node.js/Express.js backend via RESTful APIs. Data is persisted in an SQLite database managed through Prisma ORM. Authentication uses JSON Web Tokens (JWT).

---

2. Overall Description

2.1 Product Perspective

PathFinder is a standalone web application. It does not integrate with or depend on any external university systems. It uses an external LLM API for AI features but includes full mock fallbacks for operation without API keys.

2.2 Product Functions (High Level)

1. User account management (registration, login, password reset)
2. Profile management (student and organization profiles)
3. Opportunity management (CRUD operations)
4. Application lifecycle management
5. AI-powered matching and document generation
6. Administrative oversight and analytics
7. Support ticketing system

2.3 User Classes and Characteristics

User Class	Description	Technical Skill	Frequency of Use
Student	College/university student	Basic	Daily
Organization	HR/recruitment professional	Basic-Intermediate	Weekly
Administrator	Platform operations	Intermediate-Advanced	Daily
Developer	Future contributor	Advanced	As needed

2.4 Operating Environment

- **Client**: Modern web browser (Chrome 90+, Firefox 88+, Edge 90+, Safari 14+)
- **Server**: Node.js v18+ runtime
- **Database**: SQLite 3.x (via Prisma ORM)
- **Hosting**: Vercel (frontend), Render (backend)

2.5 Design and Implementation Constraints

- Must use TypeScript on both frontend and backend.
- Must use Prisma ORM for all database operations (no raw SQL).
- Must use JWT for authentication (no session-based auth).
- Must operate within free-tier hosting limits.

---

3. Functional Requirements

3.1 Authentication Module (FR-AUTH)

ID	Requirement	Priority
FR-AUTH-01	System SHALL allow users to register with email, password, and role selection (STUDENT or ORGANIZATION)	P0
FR-AUTH-02	System SHALL hash passwords using bcrypt with 10 salt rounds before storage	P0
FR-AUTH-03	System SHALL generate a JWT token upon successful login with 24-hour expiry	P0
FR-AUTH-04	System SHALL prevent duplicate email registrations	P0
FR-AUTH-05	System SHALL support password reset via email with secure token generation	P1
FR-AUTH-06	System SHALL invalidate reset tokens after use or after 1 hour expiry	P1
FR-AUTH-07	System SHALL apply rate limiting (5 requests per 15 minutes) to auth endpoints	P0

3.2 Student Module (FR-STUDENT)

ID	Requirement	Priority
FR-STU-01	System SHALL create a StudentProfile record upon student registration	P0
FR-STU-02	System SHALL allow students to update their profile (name, bio, major, university, graduation year)	P0
FR-STU-03	System SHALL allow students to add, update, and remove skills	P0
FR-STU-04	System SHALL allow students to add certifications with name, issuer, and date	P1
FR-STU-05	System SHALL allow students to add portfolio links (title + URL)	P1
FR-STU-06	System SHALL allow students to upload documents (resume, SOP, transcript) with 5MB file size limit	P0
FR-STU-07	System SHALL restrict file uploads to PDF, DOC, DOCX, PNG, JPG, JPEG formats	P0
FR-STU-08	System SHALL allow students to upload a profile picture	P1

3.3 Organization Module (FR-ORG)

ID	Requirement	Priority
FR-ORG-01	System SHALL create an OrganizationProfile record upon organization registration	P0
FR-ORG-02	System SHALL allow organizations to update their profile (company name, description, website, industry, logo)	P0
FR-ORG-03	System SHALL display organization profile information on opportunity listings	P0

3.4 Opportunity Module (FR-OPP)

ID	Requirement	Priority
FR-OPP-01	System SHALL allow organizations to create opportunities with title, description, category, deadline, eligibility, location, stipend, and duration	P0
FR-OPP-02	System SHALL support four opportunity categories: INTERNSHIP, SCHOLARSHIP, GRANT, HACKATHON	P0
FR-OPP-03	System SHALL support three opportunity statuses: DRAFT, PUBLISHED, ARCHIVED	P0
FR-OPP-04	System SHALL allow organizations to mark opportunities as featured	P1
FR-OPP-05	System SHALL display only PUBLISHED opportunities to students	P0
FR-OPP-06	System SHALL allow keyword search across opportunity titles and descriptions	P0
FR-OPP-07	System SHALL allow filtering by category and location	P0

3.5 Application Module (FR-APP)

ID	Requirement	Priority
FR-APP-01	System SHALL allow students to submit applications with optional cover letter	P0
FR-APP-02	System SHALL prevent duplicate applications to the same opportunity	P0
FR-APP-03	System SHALL support six application statuses: PENDING, REVIEWING, SHORTLISTED, ACCEPTED, REJECTED, WITHDRAWN	P0

FR-APP-04	System SHALL allow students to withdraw applications (except accepted ones)	P0
FR-APP-05	System SHALL create timeline events for every status change	P1
FR-APP-06	System SHALL allow organizations to update application status individually	P0
FR-APP-07	System SHALL allow organizations to batch-update multiple application statuses	P1
FR-APP-08	System SHALL send notifications to students when application status changes	P1
FR-APP-09	System SHALL calculate an AI match score (0-100) for each application	P0
FR-APP-10	System SHALL generate a human-readable match reason for each score	P0

3.6 AI Module (FR-AI)

ID	Requirement	Priority
FR-AI-01	System SHALL generate Statements of Purpose personalized to student profile and opportunity	P1
FR-AI-02	System SHALL generate Cover Letters personalized to student profile and opportunity	P1
FR-AI-03	System SHALL generate Essays with optional topic prompts	P1
FR-AI-04	System SHALL generate Personal Statements based on student profile	P1
FR-AI-05	System SHALL parse uploaded PDF resumes and extract structured profile data	P1
FR-AI-06	System SHALL calculate a profile strength/completeness score (0-100%)	P1
FR-AI-07	System SHALL provide personalized opportunity recommendations based on student skills	P1
FR-AI-08	System SHALL generate AI candidate summaries for organization review	P1
FR-AI-09	System SHALL save all AI generations to the AIGeneration table for history retrieval	P1
FR-AI-10	System SHALL provide mock fallback responses when external AI API is unavailable	P0

3.7 Notification Module (FR-NOTIF)

ID	Requirement	Priority
FR-NOTIF-01	System SHALL create notifications for application status changes	P1
FR-NOTIF-02	System SHALL create notifications for new applications received by organizations	P1
FR-NOTIF-03	System SHALL track read/unread status for each notification	P1
FR-NOTIF-04	System SHALL allow users to mark all notifications as read	P1

3.8 Support Ticketing Module (FR-TICKET)

ID	Requirement	Priority
FR-TKT-01	System SHALL allow any authenticated user to create support tickets	P1
FR-TKT-02	System SHALL support ticket types: BUG, REPORT, FEATURE_REQUEST, OTHER	P1
FR-TKT-03	System SHALL support ticket priorities: LOW, MEDIUM, HIGH	P1
FR-TKT-04	System SHALL support ticket statuses: OPEN, IN_PROGRESS, RESOLVED, CLOSED	P1
FR-TKT-05	System SHALL allow threaded messages within each ticket	P1
FR-TKT-06	System SHALL restrict ticket viewing to the creator and admins	P1
FR-TKT-07	System SHALL allow only admins to change ticket status	P1

3.9 Admin Module (FR-ADMIN)

ID	Requirement	Priority
FR-ADM-01	System SHALL provide platform analytics (total users, opportunities, applications, status breakdowns)	P1
FR-ADM-02	System SHALL allow admins to search and filter user accounts	P1
FR-ADM-03	System SHALL allow admins to change user roles	P1
FR-ADM-04	System SHALL allow admins to delete user accounts (except self)	P1
FR-ADM-05	System SHALL allow admins to view and delete any opportunity	P1
FR-ADM-06	System SHALL provide an audit log of platform actions	P1
FR-ADM-07	System SHALL provide system health information (uptime, memory, database counts)	P1
FR-ADM-08	System SHALL allow analytics export as CSV	P2

---

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses SHALL complete within 2 seconds under normal load
NFR-02	Performance	Frontend pages SHALL achieve First Contentful Paint under 1.5 seconds
NFR-03	Security	All passwords SHALL be hashed before storage (bcrypt, 10 rounds)
NFR-04	Security	All protected endpoints SHALL verify JWT tokens before processing
NFR-05	Security	File uploads SHALL be restricted by type and size (5MB max)
NFR-06	Security	Document files SHALL require authentication for access; images SHALL be publicly accessible
NFR-07	Usability	The interface SHALL support dark mode and light mode
NFR-08	Usability	All forms SHALL display validation errors inline
NFR-09	Reliability	The system SHALL include mock fallbacks for all external AI dependencies
NFR-10	Maintainability	All database operations SHALL use Prisma ORM (no raw SQL)
NFR-11	Portability	The database SHALL use SQLite for zero-configuration deployment
NFR-12	Scalability	The architecture SHALL support migration to PostgreSQL without code changes (Prisma adapter swap)

---

5. Use Cases

UC-01: Student Applies to Opportunity

- **Actor**: Student
- **Precondition**: Student is logged in and has a profile.
- **Flow**:
    1. Student navigates to opportunity listing.
    2. Student clicks "Apply."
    3. System checks for duplicate application.
    4. Student optionally writes a cover letter.
    5. System creates the Application record.
    6. System creates an SUBMITTED event in the ApplicationEvent table.
    7. System sends a notification to the organization.
    8. System asynchronously calculates the AI match score.
    9. System displays confirmation to the student.
- **Postcondition**: Application is created with PENDING status and a match score is generated.

UC-02: Organization Reviews Applicants

- **Actor**: Organization
- **Precondition**: Organization has posted at least one opportunity with applicants.
- **Flow**:
    1. Organization navigates to their dashboard.
    2. Organization selects an opportunity to view applicants.
    3. System displays applicants sorted by match score.
    4. Organization reviews candidate profile, resume, and AI summary.
    5. Organization updates status (REVIEWING, SHORTLISTED, ACCEPTED, or REJECTED).
    6. System creates a STATUS_CHANGED event.
    7. System sends a notification to the student.
- **Postcondition**: Application status is updated and the student is notified.

UC-03: Admin Resolves Support Ticket

- **Actor**: Admin
- **Precondition**: A user has submitted a support ticket.
- **Flow**:
    1. Admin navigates to the Support Tickets section.
    2. Admin views all open tickets.
    3. Admin opens a specific ticket to read the description.
    4. Admin sends a reply message.
    5. Admin updates the ticket status to RESOLVED.
- **Postcondition**: Ticket is resolved and the user can see the admin's response.

---

6. Error Handling

Error Scenario	HTTP Code	Error Message
Missing authentication token	401	"Unauthorized: No token provided"
Invalid or expired token	401	"Unauthorized: Invalid token"
Insufficient role permissions	403	"Forbidden: Insufficient permissions"
Duplicate email registration	400	"Email already exists"
Duplicate application submission	400	"Already applied to this opportunity"
Resource not found	404	"Student profile not found" / "Opportunity not found"
Invalid file type upload	400	"Invalid file type"
File too large	400	Multer default size limit error
Rate limit exceeded	429	"Too many requests, please try again later"
Server error	500	Context-specific error message

End of Software Requirements Specification